

1 Who am I? Who did it? 3
2 What is WebDAV? 4
3 Why WebDAV 5
4 How does it work? 6
5 The PEAR class 9
6 Current Status 25
7 more info 26





Quit

ApacheCon US 2003

WebDAV server implementation with Apache and PHP

Hartmut Holzgraefe

October 15, 2003



Contents

1 Who am I? Who did it? 3

2 What is WebDAV? 4

3 Why WebDAV 5

4 How does it work? 6

5 The PEAR class 9

6 Current Status 25

7 more info 26





Quit

1. Who am I? Who did it?



Name: Hartmut Holzgraefe `<hartmut@php.net>`

Working for: `media engineering gmbh (germ`

Using PHP since: October 1999

Hacking PHP since: December 1999

Original WebDAV work Christian Stocker
`<chregu@bitflux.ch>`

2. What is WebDAV?

WEB DISTRIBUTED AUTHORING (AND VERSIONING)

A HTTP 1.1 protocol extension (specified in RFC 2518) providing methods for

- filesystem operations
  `GET, PUT, COPY, MOVE, DELETE`
- meta-data access
  `PROPFIND, PROPPATCH`
- locking
  `LOCK, UNLOCK`

Versioning is not part of the original RFC 2518 specs and was added with RFC 3253 early last year







Quit

3. Why WebDAV

Why WebDAV? What were the alternatives back in 1996?

FTP insecure, no locking, no meta-data

HTTP had PUT only

FrontPage to proprietary

4. How does it work?

WebDAV adds additional methods to the HTTP protocol.

Some simple protocol parameters, like **Depth:** are passed as request headers, more complicated methods get additional request data as XML content payload.

Result data is always returned as XML content







A sample request:

```
PROPFIND  /container/ HTTP/1.1
Host: www.foo.bar
Depth: 1
Content-Type: text/xml; charset="utf-8"
Content-Length: ...

<?xml version="1.0" encoding="utf-8" ?>
<D:propfind xmlns:D="DAV:">
  <D:allprop/>
</D:propfind>
```





Quit



... and a sample reply

```
HTTP/1.1 207 Multi-Status
Content-Type: text/xml; charset="utf-8"
Content-Length: xxxx

<?xml version="1.0" encoding="utf-8" ?>
   <D:multistatus xmlns:D="DAV:">
     <D:response>
          <D:href>http://www.foo.bar/container/</D:href>
          <D:propstat>
               <D:prop xmlns:R="http://www.foo.bar/boxschema/">
                    <R:bigbox>
                         <R:BoxType>Box type A</R:BoxType>
                    </R:bigbox>
                    <R:author>
...
```





Quit



5. The PEAR class

The HTTP_WebDAV_Server class in PEAR takes care of most of the WebDAV protocol specific stuff, like:

- HTTP header parsing
- XML parsing and generation
- authentication
- type conversions
- status codes
- lock checks
- the OPTIONS reply
- known client ’issues’





Quit



- ...





Quit

Extending the PEAR class:

- WebDAV request methods are mapped to class methods of the same name
- Header data and XML payload is already parsed and gets passed to methods in a PHP array
- Methods have to write their results to the same array
- XML replies are created from the returned results







Quit

For a minimal working server you have to implement the `GET()`, `PUT()` and `PROPFIND()` methods. This server won't be RFC compliant but it should be usable with most clients anyway

A WebDAV Level 1 compliant server would also need `MKCOL()`, `DELETE()` and `PROPPATCH()`.

`MOVE()` and `COPY()` are emulated using `GET()`, `PUT()` and `DELETE()` if not implemented.

`LOCK()`, `UNLOCK()` and `CHECKLOCK()` are needed for full (level 2) RFC 2518 compliance

You may want to implement the `CHECK_AUTH()` method for user/password checking (although authentication is not part of the WebDAV specification)







Quit

How to start:

- extend the abstract HTTP_WebDAV_Server base class
- create an instance
- call the `ServerRequest()` method







Quit

```php
<?php

  require_once "HTTP/WebDAV/Server.php";

  class HTTP_WebDAV_Server_Filesystem extends HTTP_WebDAV_Server
{

     function GET(&$params) {
       ...
     }

     ...

  }

  $server = new HTTP_WebDAV_Server_Filesystem();

  $server->ServeRequest();

?>
```



The `GET()` method

`GET` gets only one parameter:

`$params['path']` the requested file path

As a successful `GET` generates output you have to generate the appropriate Content-Type:, Content-Length: and Last-Modified: headers yourself here.

In case of problems you have to return the appropriate HTTP status in `$params['status']`, possible values are 404 for 'not found' and 409 (conflict) for any other problem.





Quit







Quit

```php
<?php
  function GET(&$options) {
    $fspath = $this->base .  $options["path"];

    if (file_exists($fspath)) {
      if (!is_dir($fspath)) {
        header("Content-Type:  " .  mime_content_type($fspath);
      } else {
        header ("Content-Type:  httpd/unix-directory");
      }

      header("Last-Modified:  ".
        date("D, j M Y H:m:s ", file_mtime($fspath))."GMT");

      header("Content-Length:  ".filesize($fspath));

      readfile($fspath);
      return true;
    } else {
      return false;
    }
  }
?>
```

The `PUT()` method

`PUT` gets two parameters:

`$params['path']` the requested file path

`$params['stream']` a stream handle to read the transfered content from

It should return the status in `$params['status']`, possible values are 409 in case of problems, 201 if a new file was created or 204 if an existing file was successfully modified

```php
<?php
function put($options) {
  $fspath = $this->base .  $options["path"];

  if(!@is_dir(dirname($fspath))) {
    return "409 Conflict";
  }

  $new = !  file_exists($fspath);

  $fp = fopen($fspath, "w");
  if(is_resource($fp)
     && is_resource($options["stream"])) {
    while(!feof($options["stream"])) {
      fwrite($fp, fread($options["stream"], 4096));
    }
    fclose($fp);
  }

  return $new ?  "201 Created" :  "204 No Content";
}
?>
```







Quit

The `PROPFIND()` method gets three parameters:

`$params['path']` the requested file path

`$params['depth']` 0, 1 or Infinite

`&$files` a reference to an array to store property values for files in

The `$files` array is filled with one element per file. Each file entry itself has a `path` entry and a `props` array. The props entries are generated using the `MKPROP` method that expects the following parameter:

name the property name

value the property value

ns the property namespace (optional, defaults to `DAV:`)







Quit

```php
<?php
function propfind ($options,&$files) {
  $fspath = realpath($this->base .  $options["path"]);

  if (!file_exists($fspath)) return false;

  $files["files"] = array();
  $files["files"][] = $this->fileinfo($options["path"], $options);

  if (!empty($options["depth"]))  {
    if (substr($options["path"],-1) != "/") {
      $options["path"] .= "/";
    }
    $handle = opendir($fspath);

    while ($filename = readdir($handle)) {
      if ($filename != "." && $filename != "..") {
        $files["files"][] =
          $this->fileinfo ($options["path"].$filename, $options);
      }
    }
  }

  return true;
}
?>
```

```php
<?php
function fileinfo($uri, $options) {
  $fspath = $this->base .  $uri;

  $file = array();
  $file["path"]= $uri;

  $file["props"][] = $this->mkprop("displayname", strtoupper($uri

  $file["props"][] = $this->mkprop("creationdate", filectime($fsp
  $file["props"][] = $this->mkprop("getlastmodified", filemtime($

  if (is_dir($fspath)) {
    $file["props"][] = $this->mkprop("getcontentlength", 0);
    $file["props"][] = $this->mkprop("resourcetype", "collection"
    $file["props"][] = $this->mkprop("getcontenttype", "httpd/uni

  } else {
    $file["props"][] = $this->mkprop("resourcetype", "");
    $file["props"][] = $this->mkprop("getcontentlength", filesize
    if (is_readable($fspath)) {
      $file["props"][] = $this->mkprop("getcontenttype", mime_con
    } else {
      $file["props"][] = $this->mkprop("getcontenttype", "applica
    }
  }
```




```
  return $file;
}

?>
```





Quit







Quit

The `CHECK_AUTH()` method

This method gets three parameters:

- `auth_type` currently only 'Basic' is supported ...
- `username`
- `password`

It should just return `true` or `false`, you can also modify the 'Realm' string by setting `$this->http_auth_realm`



Lets see it in action ...





Quit







Quit

6. Current Status

The file server sample passes all tests performed by the `litmus` test suite

`http://www.webdav.org/neon/litmus/`

It has also been tested with the following clients:

- cadaver (unix command line)
- MicroSoft Web Folders
- WebDrive
- TeamDrive
- Dreamweaver MX
- MacOS/X
- Goliath for MacOS



7. more info

The source is available using the pear installer

```
pear install HTTP_WebDAV_Server
```

or under the following URL:

`http://pear.php.net/package-info.php?pacid=179`

The current version is 0.9, which is still labeled 'alpha' also it is already successfully used even in commercial projects like SixCMS ...





Quit

This presenation has ended.
Click anywhere to return.

_

